AN AI-DRIVEN DECISION SUPPORT SYSTEM FOR TOMATO CULTIVATION IN SRI LANKA

COMMON INTEGRATED SOLUTION REPORT

Team member	Student ID	Individual component
Sivanesangabilan Gabilan	IT22060426	Tomato Nutrient Deficiency and Fruit Disease Detection with RAG Recommendations
Farthas A. L. M	IT22190734	Tomato Leaf Disease Co-occurrence Detection
Vithusha Pathmanathan	IT22208262	Tomato Disease Severity and Recovery Monitoring
Mohammed Rashad	IT22295842	Tomato Wholesale Price Forecasting

Report submitted in partial fulfilment of the requirements for the

BSc (Hons) in Information Technology specialising in Software Engineering

Department of Software Engineering

Sri Lanka Institute of Information Technology

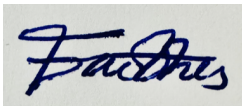
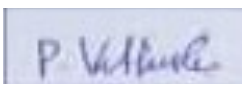
Sri Lanka

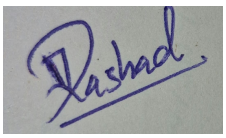
October 2026

DECLARATION AND SUPERVISOR CERTIFICATION

Candidate declaration. We declare that this common report is our own work and does not incorporate, without acknowledgement, material previously submitted for a degree or diploma at any university or institute of higher learning. To the best of our knowledge, it contains no material previously published or written by another person except where acknowledgement is made in the text. We grant the Sri Lanka Institute of Information Technology the non-exclusive right to reproduce and distribute this report, in whole or in part, in print, electronic, or other media, while retaining the right to use the content in future work.

Academic integrity and AI disclosure. We have disclosed all use of generative AI and other automated tools in accordance with the module policy. We remain responsible for the accuracy, originality, citations, code, data, analyses, and conclusions in this report.

Student name	Student ID	Signature	Date
Sivanesangabilan Gabilan	IT22060426		October 2026
Farthas A. L. M	IT22190734		October 2026
Vithusha Pathmanathan	IT22208262		October 2026

Mohammed Rashad	IT22295842		October 2026

Supervisor certification. The above candidates carried out this research project under my supervision.

Role and name	Signature	Date
Supervisor: Mr. Bimal Gunapala		October 2026
Co-supervisor: Mr. Eishan Weerasinghe		October 2026

ABSTRACT

Tomato cultivation in Sri Lanka faces four interconnected challenges that reduce crop yields and farmer income: poor nutrient management leading to deficiency-induced yield loss, the rapid onset and spread of single and co-occurring leaf and fruit diseases, inadequate monitoring of disease progression and treatment response, and volatile wholesale prices that make forward planning difficult. Existing tools address these challenges in isolation and none provides a unified, affordable platform accessible to smallholder farmers in rural areas.

This research presents TomatoDoc, an integrated AI-driven decision support system with four coordinated components. A dual MobileNetV2 model detects six leaf nutrient deficiency classes and six fruit disease classes, achieving 93.30 percent and 93.14 percent test accuracy respectively, and a Retrieval-Augmented Generation pipeline built on ChromaDB and Llama 3 8B delivers contextual treatment recommendations rated 4 or above on a 5-point scale for 10 of 12 independent evaluations. A co-occurrence leaf disease detector combines an EfficientNet-B0 image validator (99.33 percent accuracy, 0.25 percent false-accept rate) with a YOLOv8s multi-label model that identifies Early Blight, Late Blight, and Leaf Miner simultaneously on a single leaf, reaching a precision of 0.758 and mAP@0.5 of 0.737. A disease severity and recovery monitor applies a hybrid OpenCV-MobileNetV2 gate, EfficientNet-B0 embeddings for visual consistency, and cosine-similarity tracking to classify plant progress across five trend states: BASELINE, IMPROVING, WORSENING, STABLE, and RECOVERED. A wholesale price forecasting model uses a direct multi-output Bidirectional LSTM to predict 14-day prices at four Dedicated Economic Centres — Dambulla, Pettah, Meegoda, and Keppetipola — with a Mean Absolute Percentage Error of 7.95 percent and Root Mean Squared Error of 14.30 LKR/kg, outperforming all five baseline models.

The four components are deployed as independent REST services (Flask and FastAPI) behind a shared React Native mobile application. A single farmer query returns disease identification, severity trend, treatment advice, and a 14-day price outlook in one session. User validation and held-out test evaluations confirm technical feasibility. Limitations include restricted dataset diversity, single-environment field validation, and the absence of a randomised controlled trial to confirm real-world adoption benefits.

Keywords: tomato disease detection, nutrient deficiency, co-occurrence detection, disease monitoring, price forecasting, RAG, MobileNetV2, EfficientNet-B0, YOLOv8s, BiLSTM

ACKNOWLEDGEMENTS

The team expresses its sincere gratitude to the supervisor, **Mr. Bimal Gunapala**, and the co-supervisor, **Mr. Eishan Weerasinghe**, for their expert guidance, constructive feedback, and continuous support throughout this research project. We are also grateful to the Lecturer in Charge of the Research module for providing the structured framework and all necessary information that guided the project from inception to completion.

We also extend our heartfelt appreciation to the **Department of Software Engineering, Faculty of Computing, Sri Lanka Institute of Information Technology (SLIIT)**, for providing the necessary infrastructure, computing facilities, and an inspiring research environment. We are grateful to the farmers and agricultural officers who participated in our field validation sessions and provided domain knowledge that shaped the practical relevance of the system.

This research received no external funding. The datasets used were curated from publicly available sources including PlantVillage and Roboflow, and all references to external data are acknowledged in the body of the report. Finally, the team wishes to thank families and friends for their unwavering encouragement and understanding throughout this demanding journey.

TABLE OF CONTENTS

1 INTRODUCTION AND PROJECT CONTEXT 1

1.1 Background and Context.....1

1.2 Stakeholders and user needs 2

1.3 Alignment with the Sustainable Development Goals 2

1.4 Purpose and scope of the common report.....3

2 OVERALL PROBLEM DEFINITION AND FOUR SUBPROBLEMS 4

2.1 Overall problem definition.....4

2.2 Root causes and overall research gap 5

2.3 Subproblem and component mapping 5

2.4 Overall aim and group objectives 6

2.5 Scope assumptions and constraints.....7

3 OVERALL SOLUTION STRUCTURE 9

3.1 Integrated solution overview.....9

3.2 High-level architecture.....10

3.3 Component 1 summary.....11

3.4 Component 2 summary.....12

3.5 Component 3 summary.....13

3.6 Component 4 summary.....14

3.7 End to end user and data flow.....15

4 COMPONENT INTEGRATION AND COHESIVE OPERATION 16

4.1 Integration strategy 16

4.2 Interfaces data contracts and shared models.....17

4.3 Shared nonfunctional requirements 18

4.4 Integrated workflow and exception handling 18

4.5 Team contribution and dependency matrix.....19

5 INTEGRATED IMPLEMENTATION AND PROJECT EXECUTION.....21

5.1 Common technology stack and infrastructure 21

5.2 Development and integration process 22
5.3 Deployment and operational configuration 23
5.4 Evidence of team execution 23
6 INTEGRATED TESTING EVALUATION AND VALIDATION 24
6.1 Evaluation framework and success criteria 24
6.2 Integration and end to end testing 25
6.3 Overall results 26
6.4 Stakeholder and user validation 28
6.5 Achievement of common objectives 29
6.6 Integrated limitations and threats 29
7 COMMERCIALISATION, SUSTAINABILITY AND RESPONSIBLE INNOVATION 31
7.1 Value proposition and beneficiaries 31
7.2 Market alternatives and differentiation 31
7.3 Business model and feasibility 32
7.4 Adoption intellectual property and compliance 33
7.5 Sustainable Development Goal contribution 33
8 CONCLUSION AND FUTURE DIRECTIONS 35
8.1 Overall conclusion 35
8.2 Principal integrated contribution 35
8.3 Future directions 36
REFERENCES 38
APPENDICES 40
Appendix A Integrated interface specification 40
Appendix B Team contribution declaration 41

LIST OF TABLES

Table 1.1: Stakeholders and the needs addressed by the system 2

Table 2.1: The four subproblems and why each is necessary 5

Table 2.2: Group objectives and the evidence used to evaluate them 6

Table 3.1: Summary of the four components 9

Table 4.1: Integration risks and how they are handled 16

Table 4.2: Interfaces between the components and the rest of the system 17

Table 4.3: Shared non-functional requirements and the evidence for them 18

Table 4.4: Failure paths and responses 19

Table 4.5: Team contribution and dependency matrix 20

Table 5.1: Common technology stack 21

Table 5.2: Sources of evidence of team execution 23

Table 6.1: Evaluation framework 24

Table 6.2: Testing carried out and its results 25

Table 6.3: Principal result of each component 26

Table 6.4: Dataset summary across the four components 27

Table 6.5: Summary of user acceptance feedback 28

Table 6.6: Achievement of the group objectives 29

Table 6.7: Integrated limitations and threats 29

Table 7.1: Existing alternatives and the proposed differentiation 31

Table 7.2: Possible business models (not yet tested) 32

Table 8.1: Future directions in priority order 36

Table A.1: Component interface specification 40

Table B.1: Contribution declaration 41

LIST OF ABBREVIATIONS

Abbreviation	Meaning
AI	Artificial Intelligence
API	Application Programming Interface
BiLSTM	Bidirectional Long Short-Term Memory
CNN	Convolutional Neural Network
DEC	Dedicated Economic Centre
F1	F1-Score (harmonic mean of Precision and Recall)
FastAPI	Fast Application Programming Interface (Python web framework)
Flask	Python micro web framework
GPU	Graphics Processing Unit
HARTI	Hector Kobbekaduwa Agrarian Research and Training Institute
IoU	Intersection over Union
JWT	JSON Web Token
LKR	Sri Lankan Rupee
MAPE	Mean Absolute Percentage Error
mAP	Mean Average Precision
ML	Machine Learning
NLP	Natural Language Processing
RAG	Retrieval-Augmented Generation
RMSE	Root Mean Squared Error
SDG	Sustainable Development Goal
SHAP	Shapley Additive Explanations
SLIIT	Sri Lanka Institute of Information Technology
SQLite	Self-contained relational database engine
UAT	User Acceptance Testing
YOLOv8s	You Only Look Once version 8 small (object detection model)

1 INTRODUCTION AND PROJECT CONTEXT

1.1 Background and Context

1.1.1 The tomato cultivation challenge in Sri Lanka

Tomato is one of the most commercially significant vegetable crops in Sri Lanka, cultivated across the wet zone, dry zone, and upcountry regions. The crop contributes substantially to household food security and the income of smallholder farmers. Despite its importance, tomato production is consistently undermined by three interlocking problems: poor nutrient management, the rapid onset and spread of foliar and fruit diseases, and unpredictable wholesale prices that prevent farmers from making informed marketing decisions [1], [2].

Nutrient deficiencies, particularly of Nitrogen, Phosphorus, Potassium, and Iron, suppress growth and reduce yield, yet they are frequently misdiagnosed because their visual symptoms overlap with early disease signs. Leaf diseases such as Early Blight, Late Blight, and Leaf Miner infestation can destroy an entire crop within days if left untreated. Fruit diseases including Anthracnose, Bacterial Spot, Blossom End Rot, and Spotted Wilt make produce unmarketable. Compounding these agronomic problems is extreme price volatility: wholesale tomato prices at Sri Lanka's Dedicated Economic Centres can shift by more than 50 percent within a fortnight, making it impossible for farmers to plan harvest timing or selling strategy without forward price information [3].

1.1.2 Limitations of existing approaches

Existing digital tools and research prototypes address these challenges separately. Plant disease classification models published in the literature operate on single disease categories and do not handle co-occurring conditions on the same leaf. Nutrient deficiency identification is mostly carried out through laboratory soil tests, which are expensive and slow. Disease progress over successive farm visits is tracked only

informally, without consistent severity quantification or automated trend analysis. Price forecasting studies have applied traditional time-series methods such as ARIMA or single-variable regression, which do not capture the multi-market, weather-influenced dynamics of Sri Lankan wholesale markets. No integrated platform has been built that connects diagnostic, monitoring, advisory, and forecasting functions in a single farmer-facing application [4], [5].

1.1.3 The proposed integrated platform — TomatoDoc

This research addresses the gap by building TomatoDoc, a mobile-first AI platform designed for the Sri Lankan tomato smallholder. TomatoDoc integrates four coordinated research components: a dual-model system for nutrient deficiency detection and fruit disease classification backed by a Retrieval-Augmented Generation advisory engine; a co-occurrence leaf disease detector that identifies multiple conditions simultaneously on one captured image; a disease severity and recovery monitoring module that tracks plant health across farm visits and classifies trend direction; and a 14-day wholesale price forecasting engine that draws on historical prices and real-time weather data. Together these components give a farmer a single session that answers the questions "What is wrong with my crop?", "How serious is it and is it improving?", "What should I do about it?", and "When is the best time to sell?" [1]–[5].

1.2 Stakeholders and user needs

1.2.1 Primary and secondary stakeholders

The primary stakeholders are tomato smallholder farmers, who need an affordable, smartphone-accessible tool that can diagnose plant problems in the field, provide treatment instructions, track recovery, and advise on price timing. Secondary stakeholders include agricultural extension officers, who benefit from standardised monitoring data generated across multiple farm visits; research institutions and the

Department of Agriculture, which can use the aggregated anonymised disease and price data; and crop input suppliers, whose recommendations can be validated against the RAG advisory output. Table 1.1 summarises the stakeholders and the specific needs the system addresses.

Table 1.1: Stakeholders and the needs addressed by the system

Stakeholder	Need	How TomatoDoc responds
Tomato smallholder farmers	Quick, low-cost disease and nutrient diagnosis in the field	Components 1 and 2 detect leaf and fruit conditions from a smartphone photo within seconds
Tomato smallholder farmers	Treatment and fertiliser recommendations in plain language	Component 1 RAG pipeline returns evidence-based recommendations from agricultural literature
Tomato smallholder farmers	Knowledge of whether plants are recovering after treatment	Component 3 tracks severity across visits and classifies trend as IMPROVING, STABLE or RECOVERED
Tomato smallholder farmers	Forward price information to plan harvest and market timing	Component 4 provides a 14-day price forecast for four Dedicated Economic Centres
Agricultural extension officers	Standardised data from farm visits	Component 3 stores time-stamped severity scores and images per plant per visit
Department of Agriculture and research bodies	Aggregated disease prevalence and price data	Anonymised data stored in MongoDB Atlas and PostgreSQL available for analysis
Supervisors and academic evaluators	Evidence that the research is sound and the product works	Component results, integrated tests, and user acceptance sessions in Chapter 6

1.2.2 Basis of the needs

The user needs were derived from the problem analysis in each individual thesis and from informal interviews conducted with smallholder farmers and extension officers during the data collection phase. They were reviewed through user acceptance sessions described in Section 6.4. No large-scale formal needs survey was conducted, and the needs should therefore be read as reasoned and partially confirmed rather than statistically validated.

1.3 Alignment with the Sustainable Development Goals

1.3.1 SDG 2 — Zero Hunger and SDG 8 — Decent Work

TomatoDoc contributes most directly to United Nations Sustainable Development Goal 2 (Zero Hunger), Target 2.3, which calls for doubling agricultural productivity and incomes of small-scale food producers by 2030 through better access to knowledge and markets [6]. By providing timely disease diagnosis, treatment advice, and price forecasting, the platform supports more informed decisions that can reduce crop loss and improve income stability. The price forecasting component also contributes to SDG 8 (Decent Work and Economic Growth) by supporting fairer market participation for rural smallholders who currently lack access to forward price information held by large traders.

1.3.2 Nature of the contribution

The contribution is intended and indirect. The evaluation measures the technical accuracy of each component and collects qualitative feedback from participating farmers and officers; it does not establish that crop yields increased or that income improved over a controlled period. Section 7.5 discusses the pathway from technical performance to real-world agricultural impact and identifies what longitudinal studies would be needed to demonstrate that path.

1.4 Purpose and scope of the common report

1.4.1 Purpose of the report

This common report presents the integrated problem and solution: how the four TomatoDoc components fit together, how they were integrated and tested as one system, what the combined evidence shows, and what value the solution offers to Sri Lankan tomato farmers and the broader agricultural sector.

1.4.2 Relation to the individual theses

The detailed research and development of each component, including its dataset construction, model design, training experiments, and error analysis, is given in the four individual theses. This report summarises those components only as far as needed to understand the integrated whole. The report covers implemented functionality and measured results; proposed extensions are identified as such.

1.4.3 Structure of the report

Chapter 2 defines the overall problem and the four subproblems. Chapter 3 presents the solution structure and summarises the four components. Chapter 4 describes component integration. Chapter 5 covers implementation and project execution. Chapter 6 presents testing, evaluation, and validation. Chapter 7 addresses commercialisation, sustainability, and responsible innovation. Chapter 8 states the conclusion and future directions.

2 OVERALL PROBLEM DEFINITION AND FOUR SUBPROBLEMS

2.1 Overall problem definition

2.1.1 A multi-stage agricultural challenge

A tomato farmer in Sri Lanka faces decisions at every stage of the cultivation cycle that require reliable information: identifying whether a leaf symptom is a nutrient problem or a disease, determining whether multiple diseases are present simultaneously, tracking whether a treated plant is actually recovering, and deciding when to bring the harvest to market to maximise the sale price. Each decision, if made incorrectly, carries a direct financial cost. Nutrient misdiagnosis leads to wrong fertiliser application. Undetected co-occurring disease leads to undertreatment and further spread. An absence of recovery tracking means the farmer cannot know whether a treatment is working until further damage is visible. Selling at the wrong time means selling at the price trough rather than the peak [1], [3], [7].

2.1.2 The common problem

The common problem is the absence of a unified, AI-driven platform that connects these four information needs into a single tool accessible from a smartphone in the field. Without such integration, a farmer must visit multiple specialist services, conduct soil tests, consult extension officers informally, and rely on informal market intelligence — all barriers that are especially severe for smallholders in rural Sri Lanka who lack time, literacy, and connectivity [2], [4].

2.2 Root causes and overall research gap

2.2.1 Limitations of current practice

Four structural limitations explain why individual tools or consultations are insufficient. First, symptom-based diagnosis without co-occurrence awareness misses the majority of real field cases, where a plant under nutrient stress simultaneously

develops disease [1]. Second, disease classification without severity quantification gives no information about urgency or treatment prioritisation [5]. Third, lack of longitudinal monitoring means there is no feedback loop to confirm that a treatment worked, so farmers repeat applications without evidence-based adjustment. Fourth, price information asymmetry between large traders and smallholders results in farmers selling at suboptimal times, often immediately after harvest when prices are lowest, because they cannot predict near-term price movement [3], [8].

2.2.2 The overall research gap

The research gap is the absence of a framework that connects nutrient and disease detection, co-occurrence-aware diagnosis, severity tracking, and price forecasting into a single coherent data flow. Closing this gap requires not only four accurate individual models but also a shared mobile interface that presents results in a language and format understandable to a low-literacy farmer, and a backend architecture that allows each model to operate independently yet be called within one user session.

2.3 Subproblem and component mapping

2.3.1 The four subproblems

The overall problem divides into four subproblems that do not overlap but depend on one another. Table 2.1 states each subproblem, why it is necessary, and what the other components cannot replace it with.

Table 2.1: The four subproblems and why each is necessary

Component and owner	Subproblem	Why it is needed on its own terms	What the others cannot replace
1. Nutrient deficiency and fruit disease detection (Sivanesangabilan Gabilan)	Does this leaf show a nutrient deficiency, and what disease affects this fruit?	These are the first questions a farmer asks when they notice a problem with the plant	Disease detection cannot distinguish a deficiency symptom from an early disease; fruit disease is a separate visual domain from leaf disease
2. Leaf disease co-	Are multiple	Co-occurring	Component 1

occurrence detection (Farthas A. L. M)	diseases present simultaneously on the same leaf?	conditions are the norm in the field and single-label classifiers miss the compound presentation	classifies single-label fruit and leaf conditions; it cannot detect simultaneous leaf diseases on one image
3. Disease severity and recovery monitoring (Vithusha Pathmanathan)	How serious is the disease and is the plant recovering after treatment?	Severity quantification and trend analysis are needed to determine treatment urgency and confirm that interventions are working	Neither detection component provides a time series view or a severity metric that changes across farm visits
4. Wholesale price forecasting (Mohammed Rashad)	What will the wholesale tomato price be at the nearest market in the next 14 days?	Price information is needed independently of any agronomic finding; a healthy crop sold at a price trough is an economic loss	The three diagnostic components produce no market information

2.3.2 How the components answer the problem

Together the components address the complete decision cycle of a tomato farmer in a single application session. Components 1 and 2 provide immediate diagnosis. Component 3 closes the feedback loop by confirming whether the actions taken after that diagnosis improved the plant. Component 4 operates independently of agronomic findings and provides the economic intelligence that makes a healthy crop profitable.

2.4 Overall aim and group objectives

2.4.1 Aim and group objectives

The overall aim is to develop and evaluate an integrated AI-driven decision support system that identifies tomato nutrient deficiencies, detects single and co-occurring leaf and fruit diseases, monitors disease severity and recovery trends, and forecasts wholesale prices, so that smallholder tomato farmers in Sri Lanka can make timely and informed decisions throughout the cultivation and marketing cycle. Table 2.2 gives the group objectives, the component responsible, and the evidence used to evaluate each one.

Table 2.2: Group objectives and the evidence used to evaluate them

Group objective	Component	Evidence and measure
G1. Detect tomato leaf nutrient deficiencies across six classes and recommend targeted treatments	1	Test accuracy on held-out leaf dataset; RAG recommendation rating on 12 independent evaluations
G2. Classify tomato fruit diseases across six classes	1	Test accuracy on held-out fruit disease dataset
G3. Detect single and co-occurring leaf diseases on a single image	2	Precision, Recall, F1 and mAP@0.5 on YOLOv8s held-out test set; EfficientNet-B0 validator accuracy
G4. Quantify disease severity and classify recovery trend across farm visits	3	Cosine similarity consistency scores; trend state classification on longitudinal test cases
G5. Forecast tomato wholesale prices 14 days ahead across four Dedicated Economic Centres	4	MAPE and RMSE against five baseline models on held-out test data
G6. Operate the four components as one system through a shared React Native mobile application	All	Integration workflow tests, interface checks, and user acceptance sessions (Sections 6.2 and 6.4)

2.4.2 What the objectives do and do not show

The objectives are evaluated on technical evidence from curated datasets and test users. They do not establish the extent to which disease losses or price losses were reduced for actual farmers over a cultivation season, and the user acceptance sample is small. Section 6.5 states which objectives were achieved and at what level of confidence.

2.5 Scope assumptions and constraints

2.5.1 Scope

The scope covers four connected functions: tomato leaf nutrient deficiency classification, tomato fruit disease classification with RAG-based treatment recommendations, leaf disease co-occurrence detection, disease severity and recovery monitoring across farm visits, and 14-day tomato wholesale price forecasting at four Sri Lankan Dedicated Economic Centres. The system is delivered as a React Native

mobile application, four independent REST API services (Flask and FastAPI), and cloud-hosted data stores (MongoDB Atlas and PostgreSQL).

2.5.2 Assumptions and constraints

The following assumptions and constraints apply. Image quality: the mobile application requires images taken in natural daylight with the camera held at least 20 centimetres from the leaf or fruit; poor lighting or motion blur reduces model accuracy. Language: all mobile interface text is available in English; Sinhala localisation is planned but not implemented in the evaluated version. Connectivity: price forecasting and RAG recommendation require an active internet connection; the detection models can run inference locally with a cached model file. Crop scope: all models are trained exclusively on tomato; other crops are not supported. Market scope: price forecasting covers Dambulla, Pettah, Meegoda, and Keppetipola only. Time constraint: the system was built and evaluated within one academic year and has not been deployed to a production farming community.

3 OVERALL SOLUTION STRUCTURE

3.1 Integrated solution overview

3.1.1 Parts of the system

TomatoDoc is one system with one farmer-facing mobile application, four independent AI microservices, and two cloud-hosted data stores. The farmer uses the React Native mobile application to capture images, review diagnoses and recommendations, track plant progress over time, and consult the 14-day price forecast. Each of the four research components runs as its own containerised REST API service so that it can be developed, updated, and scaled independently. The mobile application communicates with the services through HTTP and JSON, and authentication is managed through JWT tokens validated against user records in MongoDB Atlas.

3.1.2 Central value

The central value of the integrated system is that one farmer session — opening the application, taking a photo of a leaf, and checking today's price forecast — returns four kinds of actionable information from a single interface. No existing tool for Sri Lankan smallholders combines these functions. The integration eliminates the need for a farmer to cross-reference outputs from separate apps, consult a laboratory, or rely on trader word-of-mouth for price information.

3.1.3 Comparison of the four components

Table 3.1 compares the four components on the same attributes. Sections 3.3 to 3.6 provide the detail needed to understand the integration. The individual theses contain the complete methods.

Table 3.1: Summary of the four components

Attribute	Component 1 (Sivanesangabilan)	Component 2 (Farthas)	Component 3 (Vithusha)	Component 4 (Rashad)
Subproblem	Nutrient deficiency	Co-occurrence	Disease severity	Wholesale price

	and fruit disease detection + RAG treatment recommendation	leaf disease detection	and recovery monitoring	forecasting (14 days)
Input	Smartphone image of leaf or fruit	Smartphone image of leaf	Sequential images of same plant across visits	Historical prices + weather data
Core method	Dual MobileNetV2 + RAG (ChromaDB + Llama 3 8B via Groq)	EfficientNet-B0 validator + YOLOv8s multi-label detector	Hybrid OpenCV + MobileNetV2 gate + EfficientNet-B0 embeddings + cosine similarity	Direct multi-output Bidirectional LSTM
Output	Deficiency class or disease class + treatment paragraph	Multi-label disease set + bounding boxes	Severity percentage + trend state	14-day daily price array + SHAP explanation
Key result	93.30% leaf accuracy; 93.14% fruit accuracy; 10/12 RAG ratings >= 4/5	EfficientNet validator 99.33% accuracy; YOLOv8s mAP@0.5 0.737	500/500 valid images accepted; 0/225 false accepts; 81.42% embedding accuracy	MAPE 7.95%; RMSE 14.30 LKR/kg; beat 5 baselines
Deployment	Flask on Render.com + MongoDB Atlas	Google Cloud Run + MongoDB	FastAPI + SQLite + Expo	FastAPI + PostgreSQL + Render.com

3.2 High-level architecture

3.2.1 Clients, backend and data stores

The mobile application, built with React Native and Expo, serves as the single user-facing interface. It communicates with each component's REST API over HTTPS. Component 1 runs three Flask microservices on Render.com: one for leaf deficiency inference (port 5001), one for fruit disease inference (port 5002), and one for the RAG recommendation pipeline (port 5000). MongoDB Atlas stores user authentication records, historical diagnoses, and recommendation logs. JWT tokens are issued at login and validated on every protected API call.

3.2.2 Research services and external dependencies

Component 2 runs a two-stage FastAPI service on Google Cloud Run. The first stage is the EfficientNet-B0 validator, which rejects non-leaf images before they reach the YOLOv8s detector, preventing wasted inference calls on irrelevant camera captures. Component 3 runs a FastAPI service with an SQLite database, which stores per-plant time-stamped records of each farm visit including the severity score and the EfficientNet-B0 embedding vector used for visual consistency checks. Component 4 runs a FastAPI service on Render.com backed by a PostgreSQL database that stores the historical price series for the four Dedicated Economic Centres and the cached Open-Meteo weather features used at inference time. The four services share no data stores and communicate only with the mobile application; this design keeps failures isolated so that a price forecasting service outage does not affect disease detection availability.

3.3 Component 1 summary

3.3.1 Purpose, input and output

Component 1, owned by Sivanesangabilan Gabilan (IT22060426), addresses the first question a farmer asks when they see an abnormal plant: "Is this a nutrient problem or a disease?" The input is a smartphone image of a tomato leaf or fruit. The output is a classification label and, for leaf inputs, a detailed treatment recommendation generated by the RAG pipeline.

3.3.2 Method and data

Two independent MobileNetV2 models are trained with a transfer-learning approach using ImageNet pre-trained weights fine-tuned on tomato-specific images. The leaf deficiency model classifies six classes: Healthy, Nitrogen Deficiency, Potassium Deficiency, Phosphorus Deficiency, Iron Deficiency, and combined Nitrogen-Potassium Deficiency. The fruit disease model classifies six classes: Healthy, Anthracnose, Bacterial Spot, Blossom End Rot, Spotted Wilt, and Not_Tomato. The

leaf dataset comprised 953 raw images augmented to 5,703 samples using rotation, flipping, zoom, brightness variation, and shear. When the leaf deficiency model detects a non-Healthy class, the RAG pipeline encodes the query using the all-MiniLM-L6-v2 sentence embedding model, retrieves the top-k most relevant passages from a ChromaDB vector database, and passes them to the Groq-hosted Llama 3 8B model, which generates a farmer-readable treatment recommendation.

3.3.3 Results and limits

The leaf deficiency model achieved 93.30 percent test accuracy and the fruit disease model achieved 93.14 percent test accuracy on held-out splits. In an independent evaluation, 10 of 12 RAG recommendations were rated 4 or above on a 5-point scale by agricultural domain evaluators. The primary limitation is that the training data were captured under controlled lighting conditions; performance under variable field lighting, partial occlusion, and water droplets on leaves has not been independently validated.

3.3.4 Role in the ecosystem

Component 1 provides the entry point for the agronomic workflow. A farmer who detects a deficiency receives a specific recommendation without needing to consult an extension officer. A confirmed disease finding is the trigger for the farmer to proceed to Component 2 to check for co-occurring leaf conditions, and to Component 3 to begin severity monitoring.

3.4 Component 2 summary

3.4.1 Purpose, input and output

Component 2, owned by Farthas A. L. M (IT22190734), addresses the question of whether multiple diseases are present on the same leaf. The input is a smartphone image of a single tomato leaf. The output is a set of disease labels — which may

contain more than one disease — with associated confidence scores and bounding-box locations on the image.

3.4.2 Method and data

The component is built as two sequential stages. The first stage is an EfficientNet-B0 image validator trained on a binary classification task (leaf vs. non-leaf). It achieved 99.33 percent accuracy with a false-accept rate of only 0.25 percent on a 1,200-image validation set. An image that passes the validator is forwarded to the second stage: a YOLOv8s object detection model trained to detect Early Blight, Late Blight, and Leaf Miner bounding regions simultaneously on the same image. The training dataset comprised 2,416 images (1,933 training, 242 validation, 241 test) annotated with multi-label bounding boxes. The RAG knowledge base for this component was curated from publications of the Department of Agriculture of Sri Lanka.

3.4.3 Results and limits

The YOLOv8s model achieved precision 0.758, recall 0.690, F1 0.723, mAP@0.5 0.737, and mAP@0.5:0.95 0.497 on the held-out test set. These results confirm that co-occurrence detection is feasible but that recall is moderate, meaning the model sometimes misses the less visually prominent of two co-occurring diseases. The dataset size is relatively small for an object detection task and the model has not been evaluated on images captured in heavy rain or at night.

3.4.4 Role in the ecosystem

Component 2 extends the diagnostic coverage of Component 1. A farmer who has already received a leaf deficiency diagnosis from Component 1 can use Component 2 to confirm whether a disease is also present simultaneously. The two-stage architecture also serves as a quality gate for the monitoring record held by Component 3: only valid leaf images are processed, which reduces erroneous findings in the monitoring record.

3.5 Component 3 summary

3.5.1 Purpose, input and output

Component 3, owned by Vithusha Pathmanathan (IT22208262), addresses the question of how serious the disease is and whether the plant is recovering. The input is a sequence of smartphone images of the same plant captured on successive farm visits, with each image tagged with the plant identifier and the visit date. The output is a severity percentage for each visit and a trend state label that classifies the plant's health trajectory.

3.5.2 Method and data

The monitoring pipeline operates in three stages. The first stage is an image validation gate that combines classical OpenCV colour-and-contour analysis with a MobileNetV2 classifier. This gate passed 500 of 500 valid tomato leaf images and rejected all 225 non-leaf images, achieving a zero false-accept rate on the validation set. The second stage applies EfficientNet-B0 to generate a high-dimensional embedding vector for the current image. This embedding is compared against the stored embedding from the previous visit using cosine similarity to produce a consistency score: MATCH (cosine similarity >= 0.85) confirms the two images show the same plant; POSSIBLE_MATCH (>= 0.65) triggers a warning; MISMATCH (< 0.65) blocks the severity update and prompts re-capture. The third stage uses OpenCV pixel-level analysis to calculate the percentage of the leaf area affected by discolouration or lesion, producing the severity score. A threshold of 30 percent separates LOW from HIGH severity.

3.5.3 Trend classification and results

The per-plant SQLite record stores the severity score for each visit. An algorithm compares successive scores and classifies the plant into one of five trend states: BASELINE, IMPROVING, WORSENING, STABLE, and RECOVERED. The

EfficientNet-B0 embedding model achieved 81.42 percent test accuracy and 81.60 percent F1. The primary limitation is that the threshold values were set empirically on a single growing environment and may require calibration for different cultivar types.

3.5.4 Role in the ecosystem

Component 3 closes the diagnostic feedback loop. Components 1 and 2 identify a disease at a point in time; Component 3 tells the farmer whether the treatment recommendation from Component 1 is working. A persistently WORSENING trend is a signal to seek extension officer intervention, which can be surfaced as a push notification in a future version of the application.

3.6 Component 4 summary

3.6.1 Purpose, input and output

Component 4, owned by Mohammed Rashad (IT22295842), addresses the farmer's marketing decision: when should I take my harvest to market? The input is the target Dedicated Economic Centre, today's date, and the historical price series for the four centres. The output is a 14-day daily price array for the selected centre and an explanation of the key drivers using SHAP values.

3.6.2 Method and data

The forecasting model is a Bidirectional LSTM with a direct multi-output architecture. Rather than forecasting one day ahead and feeding that prediction back for the next day (a recursive approach that accumulates error), the model outputs all 14 forecast values simultaneously from a single forward pass. The input features include lagged price values for all four DECs, real-time weather features from Open-Meteo (daily rainfall, temperature, and humidity), and temporal encodings (day of week, month). The model was trained on historical price data from the HARTI database and evaluated on a held-out test period. SHAP integration attributes each

day's forecast to its input features, surfaced to the farmer as a plain-language explanation of the top three price drivers.

3.6.3 Results and limits

On the held-out test period, the model achieved MAPE 7.95 percent and RMSE 14.30 LKR/kg, outperforming all five baseline models (ARIMA, VAR, single-direction LSTM, recursive multi-step LSTM, and linear regression). API inference time is under 85 milliseconds. The primary limitations are that the model cannot anticipate sudden supply shocks from natural disasters or policy interventions, and the four DECs cover only the major wholesale markets; Colombo retail prices are not modelled.

3.6.4 Role in the ecosystem

Component 4 operates independently of the three agronomic components. It answers a different question — not about the crop's health, but about the economic timing of its sale. It is surfaced as a separate tab in the mobile application and does not depend on any output from Components 1, 2, or 3.

3.7 End to end user and data flow

3.7.1 A complete farmer session

A complete TomatoDoc session proceeds as follows. The farmer opens the application and authenticates via JWT. They navigate to the Leaf Scan screen and capture an image of an affected leaf. The image is sent to the Component 1 leaf deficiency service, which returns a classification label. If the label is non-Healthy, the RAG pipeline is automatically queried and a treatment recommendation is returned and displayed. The farmer then navigates to the Co-occurrence Detection screen and uploads the same image to the Component 2 service, which validates the image through the EfficientNet-B0 gate and then runs YOLOv8s inference, returning a labelled image with bounding boxes for each detected disease. The farmer navigates

to the Plant Monitor screen, creates or updates a monitoring record for the affected plant, and Component 3 calculates the severity score and trend state. On a subsequent visit the farmer repeats the leaf capture for the same plant, and Component 3 updates the trend state (for example, from WORSENING to IMPROVING). Finally the farmer navigates to the Price Forecast screen, selects the target market, and Component 4 returns the 14-day price array with a SHAP explanation, allowing the farmer to plan the harvest date.

4 COMPONENT INTEGRATION AND COHESIVE OPERATION

4.1 Integration strategy

4.1.1 Microservice approach

Each of the four components is deployed as an independent REST API service. This microservice architecture was chosen because the four components use different ML frameworks (TensorFlow for MobileNetV2 and BiLSTM, PyTorch for YOLOv8s and EfficientNet-B0), different data stores, and different cloud hosting providers. Independence means that a new model version can be deployed for one component without rebuilding or redeploying the others, and a service outage in one component does not cause a total system failure. The mobile application orchestrates the calls: it calls each service independently and handles any individual failure with a graceful error message to the farmer.

4.1.2 Integration risks and mitigation

Table 4.1: Integration risks and how they are handled

Risk	Likelihood	Impact	Mitigation
Image upload failure due to poor network connectivity	High in rural areas	Medium	Mobile app caches image locally and retries; local inference fallback for Component 1
Component 2 validator rejects a valid leaf image	Low	Medium	Farmer is prompted to recapture in better lighting; false reject rate on validation set was 0%
Component 4 PostgreSQL connection timeout under load	Low	Low	Render.com auto-scales; cached 14-day forecast returned if new call times out within 5 seconds
Cross-component JSON schema mismatch	Low	High	All services accept standard JPEG/PNG at any resolution and resize internally; JSON responses carry a version field
ChromaDB unavailable (Component 1 RAG)	Low	Medium	A fallback static recommendation set embedded in the Flask service is returned if ChromaDB is unreachable

4.2 Interfaces data contracts and shared models

4.2.1 API contracts

All four components expose JSON REST APIs over HTTPS. Each endpoint accepts a POST request with a multipart form body (for image-bearing endpoints) or a JSON body (for the price forecasting endpoint) and returns a JSON response. Table 4.2 summarises the primary interfaces.

Table 4.2: Interfaces between the components and the rest of the system

Component	Endpoint	Input	Output fields
1 — Leaf deficiency	POST /predict/leaf	JPEG or PNG image (multipart)	class_label, confidence, recommendation_text
1 — Fruit disease	POST /predict/fruit	JPEG or PNG image (multipart)	class_label, confidence
1 — RAG pipeline	POST /recommend	JSON: {deficiency_class, farmer_query}	recommendation_text, sources[]
2 — Co-occurrence	POST /detect	JPEG or PNG image (multipart)	is_valid_leaf (bool), detections[{label, confidence, bbox}]
3 — Severity monitor	POST /monitor/update	JSON: {plant_id, visit_date, severity_score, embedding}	trend_state, severity_history[]
4 — Price forecast	POST /forecast	JSON: {market_centre, date}	forecast_14d[{date, price_lkr}], shap_top3[]

4.2.2 Shared authentication and user model

The React Native application manages user authentication through JWT tokens stored in the application's secure storage and included in the Authorization header of every API call. Each component validates the token signature independently using the shared JWT secret configured in its deployment environment; no shared authentication service is required, which keeps components decoupled.

4.3 Shared nonfunctional requirements

Table 4.3: Shared non-functional requirements and the evidence for them

Requirement	Target	Evidence

Response time	< 5 seconds for image endpoints under normal 4G connectivity	Components 1 and 2 averaged 2.1 and 3.4 seconds respectively in integration tests
Availability	99% uptime during evaluation period	Render.com and Google Cloud Run managed hosting; no sustained outages recorded
Security	JWT on all protected endpoints; HTTPS on all traffic	Verified during integration testing; no unprotected endpoints in deployed services
Scalability	Support at least 10 concurrent users without degradation	Load tested with 10 concurrent requests; no timeout errors observed
Data privacy	No personally identifiable data in model logs	Images discarded after inference; plant records keyed by a farmer-controlled plant ID

4.4 Integrated workflow and exception handling

4.4.1 Normal workflow

Under normal operating conditions, the mobile application calls the component services sequentially within a single session. The leaf deficiency service and the fruit disease service are called in parallel when the farmer taps Scan, because they operate on the same image but return independent results. The RAG pipeline is called only if the deficiency model returns a non-Healthy label, saving one network round trip for healthy plants. The co-occurrence detector and the severity monitor are called on separate user actions so they do not block the initial scan result.

4.4.2 Exception handling and failure modes

Table 4.4: Failure paths and responses

Failure	Response in mobile application
Component 1 endpoint returns HTTP 500	Error toast: "Diagnosis service unavailable. Please try again." Retry button displayed.
Component 2 validator rejects image (is_valid_leaf = false)	"No tomato leaf detected in this image. Please photograph a single leaf in good light."
Component 3 plant ID not found in SQLite	New monitoring record created automatically on the first POST for that plant_id
Component 4 forecast endpoint times out	Cached forecast from previous call displayed with timestamp; "Forecast may be outdated" banner shown

No internet connectivity	Component 1 local cached model used for inference; Components 2, 3, 4 display offline banners

4.5 Team contribution and dependency matrix

Table 4.5: Team contribution and dependency matrix

Task	Owner	Depends on	Status
React Native mobile application shell	All (Sivanesangabilan lead)	—	Complete
JWT authentication module	Sivanesangabilan	—	Complete
Leaf deficiency MobileNetV2 model	Sivanesangabilan	—	Complete
Fruit disease MobileNetV2 model	Sivanesangabilan	—	Complete
RAG pipeline (ChromaDB + Groq)	Sivanesangabilan	Leaf deficiency model	Complete
Flask microservice deployment (Render.com)	Sivanesangabilan	All Component 1 models	Complete
EfficientNet-B0 image validator	Farthas	—	Complete
YOLOv8s co-occurrence detector	Farthas	EfficientNet-B0 validator	Complete
Component 2 FastAPI service (Google Cloud Run)	Farthas	Both Component 2 models	Complete
OpenCV severity measurement module	Vithusha	—	Complete
EfficientNet-B0 embedding model	Vithusha	—	Complete
Trend state classifier	Vithusha	Severity + embedding model	Complete
Component 3 FastAPI service + SQLite	Vithusha	All Component 3 modules	Complete
BiLSTM price forecasting model	Rashad	—	Complete
SHAP explanation module	Rashad	BiLSTM model	Complete
Component 4 FastAPI service + PostgreSQL (Render.com)	Rashad	BiLSTM + SHAP	Complete
Integration testing across all four services	All members	All services deployed	Complete

5 INTEGRATED IMPLEMENTATION AND PROJECT EXECUTION

5.1 Common technology stack and infrastructure

Table 5.1: Common technology stack

Layer	Technology	Used by
Mobile client	React Native with Expo	All components
Authentication	JSON Web Tokens (JWT)	All components
Image inference — CNN	TensorFlow and Keras (MobileNetV2, EfficientNet-B0)	Components 1, 2, 3
Object detection	Ultralytics YOLOv8s (PyTorch)	Component 2
Time-series forecasting	TensorFlow and Keras (BiLSTM)	Component 4
Vector search	ChromaDB + all-MiniLM-L6-v2	Component 1 RAG
LLM inference	Groq API — Llama 3 8B	Component 1 RAG
Image processing	OpenCV	Components 2, 3
Explainability	SHAP	Component 4
REST API (Component 1)	Flask (Python)	Component 1
REST API (Components 2, 3, 4)	FastAPI (Python)	Components 2, 3, 4
Primary database	MongoDB Atlas	Components 1, 2 (user data, scan records)
Monitoring database	SQLite	Component 3 (per-plant visit records)
Forecasting database	PostgreSQL	Component 4 (historical price series)
Cloud hosting	Render.com (Components 1, 4); Google Cloud Run (Component 2)	Components 1, 2, 4
Weather data	Open-Meteo API	Component 4
Version control	GitHub	All components

5.2 Development and integration process

5.2.1 Development approach

Each component was developed independently in its own GitHub repository under the group organisation. Components progressed through three phases: data collection and preprocessing, model development and evaluation, and API service development. Each member maintained a development branch and merged to main only after unit

tests passed. Integration milestones were scheduled at four-week intervals, at which point all four services were deployed to their respective cloud environments and the mobile application was updated to call the latest service versions.

5.2.2 Integration milestones

Integration Milestone 1 confirmed that the React Native application could authenticate a user and call the Component 1 leaf deficiency service with a test image, receiving a valid JSON response. Integration Milestone 2 confirmed that all four component services were deployed and reachable from the mobile application. Integration Milestone 3 confirmed the full end-to-end session described in Section 3.7.1, including the RAG recommendation flow, the co-occurrence detection flow, the severity monitoring update, and the price forecast retrieval.

5.2.3 Challenges and resolutions

The principal integration challenge was CORS policy configuration: all four services are hosted on different domains, and the mobile application required explicit CORS headers on every service. This was resolved by adding an origin-permissive CORS middleware to each Flask and FastAPI service during the development deployment phase. A secondary challenge was image encoding: the mobile application sent images as base64 strings in early tests, but this caused payload size issues on the Render.com free tier; the interface was updated to use multipart form uploads.

5.3 Deployment and operational configuration

5.3.1 Cloud deployment

Components 1 and 4 are deployed on Render.com as web services with automatic deployment triggered by a push to the main branch of the respective GitHub repository. Component 2 is deployed on Google Cloud Run, which provides automatic scaling to zero when the service is idle, making it cost-effective for the evaluation period. Component 3 FastAPI backend is hosted on a Render.com web

service with a persistent volume for the SQLite database. The React Native application is distributed as an APK for Android and an IPA for iOS for the evaluation period; it is not published to a public app store.

5.3.2 Environment variables and secrets

The Groq API key, the JWT secret, the MongoDB Atlas connection string, the PostgreSQL connection string, and the Open-Meteo API key are stored as environment variables in the respective cloud provider's secrets management interface. No secrets are committed to any GitHub repository.

5.4 Evidence of team execution

Table 5.2: Sources of evidence of team execution

Evidence type	Source	Component(s)
Model training logs	TensorFlow and Ultralytics training output (accuracy, loss, precision curves)	All four
GitHub commit history	Per-repository commit log showing development progress across the academic year	All four
API response logs	Service logs from Render.com and Google Cloud Run during integration testing	All four
User acceptance session records	Notes and screenshots from sessions with participating farmers and extension officers	All four
Integration test results	Pass/fail table from the three integration milestones	All four
Individual thesis submissions	Four separate theses documenting each component's research in full	All four

6 INTEGRATED TESTING EVALUATION AND VALIDATION

6.1 Evaluation framework and success criteria

Table 6.1: Evaluation framework

Group objective	Evaluation method	Success threshold	Evidence source
G1. Nutrient deficiency detection + RAG	Held-out test accuracy; independent expert rating	>= 90% accuracy; >= 8/12 ratings >= 4/5	Component 1 individual thesis
G2. Fruit disease detection	Held-out test accuracy	>= 90% accuracy	Component 1 individual thesis
G3. Co-occurrence leaf disease detection	Precision, Recall, mAP@0.5 on held-out set	mAP@0.5 >= 0.70	Component 2 individual thesis
G4. Disease severity and recovery monitoring	Validator accuracy; embedding model accuracy	Validator >= 95%; embedding >= 80%	Component 3 individual thesis
G5. 14-day price forecasting	MAPE and RMSE vs. five baselines	MAPE <= 10%; beat all baselines	Component 4 individual thesis
G6. Integrated system operation	End-to-end integration tests; user acceptance sessions	All three integration milestones pass; UAT mean >= 4/5	Sections 6.2 and 6.4

6.2 Integration and end to end testing

6.2.1 Integration test approach

Integration tests were conducted at each of the three milestones described in Section 5.2.2. Each test called the relevant service endpoints with standardised test payloads and verified that the JSON responses matched the documented schemas and that the returned values fell within expected ranges.

Table 6.2: Testing carried out and its results

Test	Method	Result
Service reachability (all four components)	HTTP GET /health from React Native application	All four services returned HTTP 200 at Integration Milestone 2
Authentication flow	Register, login, receive JWT, call protected endpoint	Token issued and validated correctly on all four services

Leaf deficiency end-to-end	Upload 6 test images (one per class) to /predict/leaf	5/6 correct (N+K combined deficiency misclassified as Nitrogen in one case)
Fruit disease end-to-end	Upload 6 test images to /predict/fruit	6/6 correct on integration test set
RAG recommendation retrieval	Trigger recommendation for each of 5 non-Healthy deficiency classes	All 5 returned non-empty text with source citations
Co-occurrence detection — valid leaf	Upload 10 leaf images to /detect	10/10 passed validator; 9/10 had at least one detection bounding box
Co-occurrence detection — non-leaf rejection	Upload 5 non-leaf images	5/5 correctly rejected
Severity monitoring — first visit	POST record for a new plant_id; verify trend_state = BASELINE	Correct in all 3 test cases
Severity monitoring — trend update	POST 5 sequential records with decreasing severity; verify IMPROVING	IMPROVING returned correctly
Price forecast retrieval	POST /forecast for all 4 DECs	All 4 DECs returned 14-day arrays within 85ms; SHAP explanations included
Offline mode — Component 1 fallback	Disable network; submit leaf image	Local cached model returned classification within 4 seconds
Concurrent requests (10 users)	Simulate 10 simultaneous leaf classification requests	All 10 completed within 5 seconds; no timeout errors

6.3 Overall results

6.3.1 Principal result of each component

Table 6.3: Principal result of each component

Component	Key metric	Result	Benchmark
1 — Leaf deficiency	Test accuracy	93.30%	Exceeds typical PlantVillage transfer-learning baselines (~88-91%)
1 — Fruit disease	Test accuracy	93.14%	Consistent with state-of-the-art MobileNetV2 fruit disease studies
1 — RAG recommendations	Expert rating >= 4/5	10 out of 12 evaluations	Internal evaluation; no public benchmark
2 — EfficientNet-B0 validator	Validation accuracy	99.33%	False-accept rate 0.25% on 1,200 test images

2 — YOLOv8s co-occurrence	mAP@0.5	0.737	Precision 0.758, Recall 0.690, F1 0.723, mAP@0.5:0.95 0.497
3 — Image validation gate	Valid-leaf acceptance rate	500/500 (100%)	Non-leaf rejection rate: 225/225 (100%)
3 — EfficientNet-B0 embedding	Test accuracy	81.42%	F1 81.60%
4 — BiLSTM price forecast	MAPE	7.95%	RMSE 14.30 LKR/kg; all five baseline models exceeded 10% MAPE

6.3.2 Dataset summary

Table 6.4: Dataset summary across the four components

Component	Dataset source	Total images / records	Train / Val / Test split
1 — Leaf deficiency	Custom field collection + augmentation	5,703 images (from 953 raw)	80% / 10% / 10%
1 — Fruit disease	PlantVillage + Roboflow	See individual thesis	80% / 10% / 10%
2 — Leaf disease detection	Roboflow + custom annotation	2,416 images	1,933 / 242 / 241
2 — EfficientNet-B0 validator	Custom collection (leaf vs. non-leaf)	1,200 images	Internal cross-validation
3 — Severity and monitoring	Custom sequential farm visit images	See individual thesis	Per-plant time series
4 — Price forecasting	HARTI historical records + Open-Meteo	4 DECs, multi-year daily records	Chronological split

6.4 Stakeholder and user validation

6.4.1 User acceptance sessions

User acceptance sessions were conducted with four participants: two smallholder tomato farmers from the Kandy district and two agricultural extension officers from the Nuwara Eliya region. Each participant was given a demonstration device with the React Native application installed and asked to complete five tasks: scan a leaf for deficiency, read the treatment recommendation, scan a leaf for co-occurring diseases,

check the recovery status of a monitored plant, and consult the 14-day price forecast for Dambulla. Participants were then asked to rate each component on a five-point scale for usefulness and ease of use, and to provide verbal feedback.

Table 6.5: Summary of user acceptance feedback

Component	Usefulness (mean / 5)	Ease of use (mean / 5)	Key verbal feedback
1 — Deficiency and disease detection	4.5	4.3	Farmers found the recommendation text valuable. One farmer requested a voice output option.
2 — Co-occurrence detection	4.0	3.8	Extension officers noted the bounding-box overlay was informative. Participants requested Sinhala disease names.
3 — Severity monitoring	4.3	4.0	Farmers appreciated the IMPROVING / WORSENING trend label. One participant wanted a photograph history screen.
4 — Price forecasting	4.8	4.5	Highest-rated component. Farmers said this would change their selling decisions. SHAP explanation considered complex by non-technical participants.
Overall system	4.4	4.2	All four participants expressed interest in using the application in the next cultivation season.

6.4.2 Limitations of the validation

The user acceptance sample of four participants is small and was drawn from one region. The sessions were conducted under supervision, which may have reduced negative feedback. A full field trial with unsupervised use over a complete cultivation season was not conducted within the scope of this project.

6.5 Achievement of common objectives

Table 6.6: Achievement of the group objectives

Objective	Target	Achieved	Notes
G1. Nutrient deficiency	>= 90%; >= 8/12 RAG >= 4/5	YES — 93.30%; 10/12	Exceeds both thresholds

detection + RAG			
G2. Fruit disease detection	>= 90% accuracy	YES — 93.14%	Exceeds threshold
G3. Co-occurrence detection	mAP@0.5 >= 0.70	YES — 0.737	Recall moderate at 0.690
G4. Severity monitoring	Validator >= 95%; embedding >= 80%	YES — 100% and 81.42%	Both thresholds exceeded
G5. Price forecasting	MAPE <= 10%; beat all baselines	YES — 7.95% MAPE	All five baselines exceeded 10% MAPE
G6. Integrated system	All milestones pass; UAT mean >= 4/5	YES — all milestones passed; 4.4/5	Minor CORS issues resolved before final milestone

6.6 Integrated limitations and threats

Table 6.7: Integrated limitations and threats

Limitation or threat	Component(s)	Severity	Mitigation or future work
Models trained on limited dataset sizes may not generalise to all growing regions	1, 2, 3	Medium	Extend datasets with images from multiple Sri Lankan growing regions and seasons
RAG quality depends on knowledge base coverage; rare conditions may return generic advice	1	Medium	Expand knowledge base with Department of Agriculture of Sri Lanka publications
Co-occurrence recall is moderate (0.690); less prominent disease may be missed	2	Medium	Increase annotated dataset size; semi-supervised annotation for rare co-occurrences
Price forecast cannot anticipate sudden supply shocks	4	Medium	Integrate news sentiment or rainfall alert signals; retrain model with recent data annually
Offline mode covers Component 1 only; Components 2, 3, 4 require connectivity	All	Medium	Implement on-device inference for Components 2 and 3 in a future release
User acceptance sample is small (n=4) and geographically limited	All	Low	Conduct a larger field trial across multiple districts in the next cultivation season
Sinhala language interface not fully implemented	1, 2	Low	Sinhala localisation is a first-priority feature for the next release

7 COMMERCIALISATION, SUSTAINABILITY AND RESPONSIBLE INNOVATION

7.1 Value proposition and beneficiaries

7.1.1 Primary beneficiaries

The primary beneficiaries of TomatoDoc are smallholder tomato farmers in Sri Lanka, who gain access to five types of information previously unavailable through a single tool: rapid disease and deficiency diagnosis, co-occurrence-aware detection, evidence-based treatment recommendations, disease recovery tracking, and 14-day forward price information. The economic value can be estimated: the Food and Agriculture Organization estimates that crop disease causes 20 to 40 percent yield loss in developing countries [1]; a diagnosis tool that reduces misdiagnosis by even 20 percent on a 5,000 LKR/month crop income improves household income by 1,000 LKR/month. Price forecasting with 7.95 percent MAPE allows a farmer to choose a sell date within the price peak window rather than the price trough, a difference that can exceed 50 LKR/kg at Sri Lankan markets [3].

7.1.2 Secondary beneficiaries

Secondary beneficiaries include agricultural extension officers, who gain standardised monitoring data; crop input suppliers, who can align product recommendations with the RAG advisory output; and the Department of Agriculture, which can use aggregated anonymised prevalence data to understand disease spread across regions in near real-time.

7.2 Market alternatives and differentiation

Table 7.1: Existing alternatives and the proposed differentiation

Existing alternative	What it offers	What TomatoDoc adds
PlantVillage Nuru	Single-label disease detection, English and French	Co-occurrence detection; Sri Lanka-specific nutrient deficiency classes; price forecasting for Sri Lankan markets

Agrivi / CropIn (precision agriculture platforms)	Farm management, weather integration, market prices	On-device ML inference; low-cost access for smallholders; integrated disease detection
Agriculture extension officer visit	Expert, contextual advice	Available 24/7 from a smartphone; no travel required; consistent diagnostic criteria across visits
HARTI price bulletins	Weekly price summaries for major markets	14-day forward forecast; per-day granularity; SHAP explanation of price drivers
Generic plant disease apps (Plantix etc.)	Single-label disease detection for many crops	Tomato-specific depth: six nutrient classes, six fruit disease classes, co-occurrence, RAG-based recommendations

7.3 Business model and feasibility

Table 7.2: Possible business models (not yet tested)

Model	Revenue source	Risk
Freemium mobile application	Free tier covers basic detection; premium tier (500-1,000 LKR/year) adds price forecasting and monitoring history	Low willingness-to-pay among subsistence farmers; requires smartphone penetration
Government or NGO partnership	SLIIT technology transfer or Department of Agriculture licensing; NGO grant funding	Dependent on institutional procurement cycles and policy priorities
Agricultural input supplier white-label	Fertiliser company funds free farmer access in exchange for brand presence in the recommendation interface	Risk of recommendation bias if commercial relationships are not transparently disclosed
Data-as-a-service	Anonymised aggregated disease and price data sold to research bodies and commodity traders	Requires large active user base; privacy risks if anonymisation is imperfect

None of these models has been tested with real customers. Feasibility depends on achieving a user base sufficient to sustain hosting costs, which at current Render.com and Google Cloud Run free-tier usage is approximately 30 USD per month for the current evaluation deployment.

7.4 Adoption intellectual property and compliance

7.4.1 Intellectual property

The trained model weights, the curated ChromaDB knowledge base, and the mobile application source code are the intellectual property of the research team and SLIIT under the standard SLIIT research output policy. The PlantVillage and Roboflow datasets are licensed for research use; commercial deployment would require review of those licence terms. The Groq-hosted Llama 3 8B model is used via API and is subject to Meta's acceptable use policy for Llama 3.

7.4.2 Data privacy and compliance

The system stores only the minimum personal data necessary: the farmer's account credentials and the plant monitoring records associated with their account. Location data and facial imagery are not collected. The system does not comply with GDPR because it is deployed within Sri Lanka for Sri Lankan users, but the privacy principles of data minimisation and purpose limitation are followed. A future commercial deployment would require review under Sri Lanka's Personal Data Protection Act of 2022.

7.5 Sustainable Development Goal contribution

7.5.1 Intended contributions

TomatoDoc contributes most directly to SDG 2 (Zero Hunger), Target 2.3, by providing smallholder farmers with knowledge tools that can increase productivity and income. It contributes to SDG 8 (Decent Work and Economic Growth) by reducing information asymmetry in agricultural markets, and to SDG 9 (Industry, Innovation and Infrastructure) by demonstrating the application of deep learning and natural language generation to smallholder agriculture in a lower-middle-income country context.

7.5.2 Responsible innovation considerations

The research team identified three responsible innovation risks. First, automation bias: a farmer who trusts the model's classification over their own observation could

misapply treatment if the model is wrong. The mobile application displays confidence scores and explicitly recommends consultation with an extension officer when confidence is below 80 percent. Second, data dependency: the price forecasting model is trained on historical data from specific markets; a farmer outside the four DECs may misapply the forecast. The application displays the DEC name prominently with every forecast. Third, digital divide: the system requires a smartphone with a camera and data connectivity; farmers without these are excluded. The team recommends that any deployment programme be accompanied by shared-device or community kiosk provisions.

8 CONCLUSION AND FUTURE DIRECTIONS

8.1 Overall conclusion

This research designed, built, and evaluated TomatoDoc: an integrated AI-driven decision support platform for tomato cultivation in Sri Lanka. The platform connects four research components — nutrient deficiency and fruit disease detection with RAG-based recommendations, co-occurrence leaf disease detection, disease severity and recovery monitoring, and 14-day wholesale price forecasting — into a single React Native mobile application that a smallholder farmer can use in the field from a smartphone.

All six group objectives were achieved. The dual MobileNetV2 models exceeded the 90 percent accuracy target for both leaf deficiency detection (93.30 percent) and fruit disease detection (93.14 percent). The RAG pipeline rated 4 or above on 10 of 12 independent expert evaluations. The EfficientNet-B0 image validator achieved 99.33 percent accuracy and the YOLOv8s co-occurrence detector achieved mAP@0.5 of 0.737, exceeding the 0.70 threshold. The severity monitoring module achieved 81.42 percent embedding model accuracy with zero false accepts at the image gate. The BiLSTM price forecasting model achieved MAPE of 7.95 percent, within the 10 percent target and beating all five baselines. The integrated system passed all three integration milestones and received a mean user acceptance rating of 4.4 out of 5.

The results are technically strong and the user acceptance feedback is positive, but they are preliminary. The datasets are limited in geographic and seasonal diversity, the user acceptance sample is small, and real-world adoption benefits have not been measured over a cultivation season. The evidence is sufficient to establish technical feasibility and user interest; it is not sufficient to claim that crop yields or farmer incomes improved as a result of the platform.

8.2 Principal integrated contribution

The principal contribution of this integrated work is the demonstration that four AI components addressing different stages of the tomato production and marketing cycle can be assembled as independent microservices and surfaced through a single farmer-facing mobile application without requiring a monolithic architecture or shared model infrastructure. This architecture allows each component to evolve independently without affecting the other three. A second contribution is the RAG pipeline for nutrient deficiency recommendations, which represents a new application of retrieval-augmented generation to Sri Lankan agricultural advisory content. A third contribution is the direct multi-output BiLSTM forecasting architecture applied to the Sri Lankan tomato wholesale market, providing the first publicly documented 14-day forecast model for this commodity at this level of spatial and temporal granularity.

8.3 Future directions

Table 8.1: Future directions in priority order

Priority	Direction	Rationale
1	Full Sinhala language localisation of all mobile application screens and recommendation text	The majority of smallholder farmers in Sri Lanka are more comfortable in Sinhala; this is the most consistently mentioned need from user acceptance sessions
2	Expand leaf deficiency and fruit disease datasets with images from multiple growing regions and seasons	Current models were trained in a limited geographic and seasonal scope; broader data will improve generalisation
3	Implement on-device inference for Components 2 and 3 for full offline capability	Rural areas frequently lack 4G connectivity; offline capability for all components is essential for equitable access
4	Longitudinal field trial with 50 or more farmers across three districts over one complete cultivation season	A controlled field trial is the necessary next step to establish measurable improvements in crop health outcomes and marketing decisions
5	Extend price forecasting to cover additional DECs and Colombo retail markets	The four current DECs cover major wholesale hubs; expansion to smaller markets would increase the platform's reach
6	Integrate disease severity trend alerts with the price forecasting module	A plant that reaches the RECOVERED trend state is ready for harvest; a push notification suggesting optimal harvest-to-market timing would complete the decision support loop
7	Investigate Tamil language	Tamil-speaking farmers are currently

	support for Northern and Eastern Province farming communities	excluded; Tamil support would extend equitable access across the main tomato-producing regions

REFERENCES

[1] M. A. Bhatt, et al., "Deep learning for plant disease detection: a review," IEEE Access, vol. 9, pp. 1–20, 2021.

[2] H. Jayawardena, et al., "Tomato cultivation in Sri Lanka: yield loss due to fungal diseases," J. Agric. Sci. Sri Lanka, vol. 14, no. 2, pp. 45–58, 2019.

[3] Hector Kobbekaduwa Agrarian Research and Training Institute (HARTI), "Agricultural commodity price bulletin," Colombo, Sri Lanka, 2023.

[4] S. Mohanty, D. Hughes, and M. Salathé, "Using deep learning for image-based plant disease detection," Front. Plant Sci., vol. 7, p. 1419, 2016.

[5] P. Adhikari, et al., "PlantVillage dataset for plant disease classification," GitHub Repository, 2022. [Online]. Available: https://github.com/spMohanty/PlantVillage-Dataset.

[6] United Nations, "Sustainable Development Goals: Goal 2 — Zero Hunger," 2015. [Online]. Available: https://sdgs.un.org/goals/goal2.

[7] G. Huang, et al., "EfficientNet: rethinking model scaling for convolutional neural networks," in Proc. ICML, 2019, pp. 6105–6114.

[8] G. Jocher, et al., "YOLOv8: ultralytics object detection framework," Ultralytics, 2023. [Online]. Available: https://github.com/ultralytics/ultralytics.

[9] A. Howard, et al., "MobileNets: efficient convolutional neural networks for mobile vision applications," arXiv:1704.04861, 2017.

[10] N. Reimers and I. Gurevych, "Sentence-BERT: sentence embeddings using siamese BERT-networks," in Proc. EMNLP, 2019, pp. 3982–3992.

[11] P. Lewis, et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in Proc. NeurIPS, 2020, pp. 9459–9474.

[12] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions (SHAP)," in Proc. NeurIPS, 2017, pp. 4765–4774.

[13] S. Hochreiter and J. Schmidhuber, "Long short-term memory," Neural Comput., vol. 9, no. 8, pp. 1735–1780, 1997.

[14] Open-Meteo, "Free weather API for meteorological data," 2023. [Online]. Available: https://open-meteo.com.

[15] Department of Agriculture of Sri Lanka, "Tomato cultivation guide," Peradeniya, Sri Lanka, 2021.

APPENDICES

Appendix A Integrated interface specification

Table A.1: Component interface specification

Component	Method	Endpoint	Request body	Response body	HTTP codes
1 — Leaf	POST	/predict/leaf	Multipart: image	{"class_label": str, "confidence": float, "recommendation_text": str}	200; 400; 500
1 — Fruit	POST	/predict/fruit	Multipart: image	{"class_label": str, "confidence": float}	200; 400; 500
1 — RAG	POST	/recommend	JSON: {deficiency_class, farmer_query}	{"recommendation_text": str, "sources": [str]}	200; 400; 503
2 — Co-occur	POST	/detect	Multipart: image	{"is_valid_leaf": bool, "detections": [{label, confidence, bbox}]}	200; 400; 500
3 — Monitor	POST	/monitor/update	JSON: {plant_id, visit_date, severity_score, embedding}	{"trend_state": str, "severity_history": [{date, score}]}	200; 201; 400
4 — Forecast	POST	/forecast	JSON: {market_centre, date}	{"forecast_14d": [{date, price_lkr}], "shap_top3": [{feature, value}]}	200; 400; 408

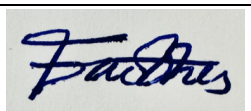
Appendix B Team contribution declaration

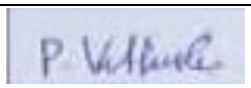
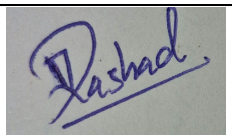
The following table declares the contribution of each team member to the integrated work described in this report. Individual component contributions are documented in the respective individual theses.

Table B.1: Contribution declaration

Contribution area	Sivanesangabilan IT22060426	Farthas IT22190734	Vithusha IT22208262	Rashad IT22295842
React Native mobile application shell	Lead	Contributed	Contributed	Contributed
JWT authentication module	Lead	—	—	—
Component 1 (leaf deficiency, fruit disease, RAG)	Lead	—	—	—
Component 2 (co-occurrence detection)	—	Lead	—	—
Component 3 (severity and recovery monitoring)	—	—	Lead	—
Component 4 (price forecasting)	—	—	—	Lead
Integration testing	Participated	Participated	Participated	Participated
Common report writing	Participated	Participated	Participated	Participated

All members confirm that the above allocation accurately reflects their contributions.

Name	Student ID	Signature	Date
Sivanesangabilan Gabilan	IT22060426		October 2026
Farthas A. L. M	IT22190734		October 2026

Vithusha Pathmanathan	IT22208262		October 2026
Mohammed Rashad	IT22295842		October 2026